

Solutions to the Traveling Salesman Problem

Omar Attaalla - 37494662

March 2024

1 Introduction

The Traveling Salesman Problem (TSP) is a famous Computer Science problem that has been researched for decades. This problem is particularly difficult to solve in polynomial time and belongs to the set NP-Hard (That is, a reduction exists from the TSP to a problem in NP Hard). No complete and optimal solution that executes in polynomial time has ever been discovered. The problem goes as follows: Given a set of cities, where there is a path between every city, and the distances between every pair of cities, find the shortest path that visits every city once and returns to the origin city.

In this paper, we will discuss two different solutions to the Traveling Salesman problem. One solution is complete and optimal and uses a branch and bound method with a distance based heuristic. The other solution is not complete/optimal but runs much quicker than the latter. The second solution computes a path that is close to the optimal shortest path.

We will primarily focus on the performance aspect of the discussed solutions to the TSP. In particular, we will discuss when a certain solution works best, why in certain scenarios a solution may not work, and the overall runtime of each solution.

2 Branch and Bound

The first solution uses the branch and bound technique for solving combinatorial optimization problems. We will represent the TSP as a graph where each node is a city and each edge between two cities is weighted with the distance between the two cities. In particular, we will store the problem as an adjacency matrix.

2.1 Assumptions

1. We assume that the TSP is symmetric such that the distance between city i and city j is the same in both directions.
2. We assume the graph is complete, that is there is an edge from every city to every other city.

We can compute all the possible paths for the graph while keeping track of the best path we find. We can do this by using a recursive DFS algorithm through the graph. Starting at each node, we can choose from the set of the remaining nodes (generate all possible permutations to travel the city). This process is exponential in terms of running time since we generate all the possible paths for the graph.

2.2 Lower Bound Heuristic Pruning

We can make a slight improvement by pruning the search space. Since we keep track of the current best path cost we have found, there is no reason to traverse a path that has exceeded in cost to our best path. We can go further and compute a lower bound on the remaining path cost to see if a path is worthwhile. We will use this lower bound remaining path cost as our heuristic to eliminate paths that cannot improve our current best. In order to compute the heuristic, we need to find the minimum cost path from the remaining cities. We find the lower bound heuristic by finding the minimum edge entering and exiting each remaining city (That is, we find the two lowest cost edges incident to a city). This branch and bound method is discussed in [1]. The pseudocode of the heuristic for a given node is shown below:

```
lowerBoundNode(M[][], node, availableNodes[], size) {  
    min1 = inf  
    min2 = inf  
  
    for ( i = 0; i < size; i++) {  
        if ( i == node ) {
```

```

        continue;
    }

    if (M[nodeIndex][i] < min1 && availableNodes[i]) {
        min2 = min1;
        min1 = M[nodeIndex][i];
    } else if (M[nodeIndex][i] < min2 && availableNodes[i]) {
        min2 = M[nodeIndex][i];
    }
}

if (min1 == inf or min2 == inf) { //leaf node
    return 0;
}

return (min1 + min2) / 2;
}

```

The heuristic starts by initializing two variables to infinity (that we will minimize). We denote the adjacency matrix as the 2d array `M`. We then traverse every edge incident to the node until we find the two least cost nodes. If the minimum remains at infinity, we have a leaf node. Finally, the algorithm returns the two closest cities divided by 2.

The complete heuristic computes the two closest cities for each available city. The psudeocode, that uses the above `lowerBoundNode` function is shown below:

```

lowerBoundGraph(M[][] , availableNodes[] , size) {
    lowerBound = 0;

    for ( i = 0; i < size; i++) {
        lowerBound += lowerBoundNode(M, i , availableNodes , size);
    }

    return lowerBound;
}

```

The complete heuristic sums the `lowerBoundNode` value for each node. This results in the summation of the cheapest two cities from each available node divided by two.

2.3 Correctness and Consistency

Since the `lowerBoundGraph` heuristic is by definition a lower bound, we never overestimate in the search space. This shows that the heuristic is consistent and admissible. A solution to the TSP will contain edges with a degree of 2 (one edge entering a city and one edge exiting the city). If we take the two lowest cost edges that are incident to a node, we are guaranteed to have a lower bound on the cost to goal. Therefore, this heuristic is consistent and therefore admissible. Since the heuristic is consistent, we can conclude that the algorithm will return an optimal TSP solution. A more detailed proof is discussed in [1].

2.4 The Complete Algorithm

The complete algorithm utilizes the above data structures as well as the previously defined heuristic to perform the branch and bound on the search space. We will define a recursive `branchAndBound` function that repeatedly takes the minimum cost move from the priority queue (the search space) and prunes branches that exceed our upper bound. When a leaf node is found, we will set our upper node to the cost of that path.

The branchAndBound function utilizes the adjacency matrix and the priority queue currentMoves. It starts by getting the cheapest cost "move" from the priority queue. This returns a Node that represents a reverse linked list. We then mark the availableMoves that are remaining in the current path we just selected. We check the lower bound of the remaining moves to see if it exceeds our upper bound "best". If it does exceed, we prune the search space. We then check if the currentNode is a leaf node. If it is a leaf, we can see if the total cost of the path is better than our upper bound. If it is better than our upper bound we can update our best path to the current path and the upper bound to our current path. Finally, we expand the current node to the possible moves we can make that has the possibility of being better than our upper bound. We then recurse to the next move.

The above function needs a starting node to expand. We define an upper function lowerBoundTSP that will serve as the main function. It iterates through every starting node and calls the above branch and bound function. The pseudocode is shown below:

```
lowerBoundTSP() {
    remainingNodes = int[numVertices];
    for (i = 0; i < numVertices; i++) {
        remainingNodes[i] = 1;
    }

    Node* best = nullptr; //the best path
    for (i = 0; i < numVertices; i++) {
        // remainingNodes[i] = 0;
        p-q <Node> currentMoves;
        start = Node(i,
                    lowerBoundGraph(adjMatrix, remainingNodes, numVertices),
                    nullptr, i);
        currentMoves.push(newNode); // push the first node
        branchAndBound(currentMoves, best);
        remainingNodes[i] = 1;
    }

    return best.getPath();
}
```

We start by initializing an array that denotes the remaining cities we can choose. We then iterate over every city and set it as the starting city. We run the branch and bound function on every starting city and we return the best path. The getPath method traverses the reverse linked list Node and returns the final path.

2.5 Space and Time Complexity

To analyze the time complexity, we will take a look at every method. To compute our heuristic we iterate over the entire graph, specifically we visit every edge of the graph. Therefore, the heuristic is computed in $O(E)$ time where $E = N \cdot (N - 1) = O(N^2)$ since the graph is complete.

The branch and bound can have a worst case runtime of $O(N!)$. This is because it is possible that no pruning happens. However, in practice pruning is likely to occur. Since we compute the heuristic in every iteration, we will have a time complexity of $O(N^2 \cdot N!)$.

The worst case space complexity of the branch and bound algorithm is dominated by the priority queue. This queue can grow to a factor of $B \cdot D$ where B is the branching factor ($N!$ in the worst case) and D is the depth (N). Therefore, the space complexity is $O(N \cdot N!)$. The search space reduces the number of choices at each level by 1, which is why the branching factor is $O(N!)$.

2.6 Experimental Approach

To measure the performance in practice, we run multiple tests for various different city lengths. For each problem, we have variables k, u, and v. The variable k refers to the number of distinct values used to generate the Graph. The variable u refers to the mean of the normal distribution. The variable v refers to the standard deviation of the normal distribution for the distances. Each problem is generated with a different number of cities and different values for k, u, and v.

By varying the parameters of each problem, we are able to see which variables effect the running time of the algorithm. In particular, we are able to notice patterns in the results. When computing an average or an entry in one of the tables below, we used hundreds of problems of the same parameters to get a much wider and applicable data set.

2.6.1 Experimental Results

Since Branch and Bound is heavily dependent on the number of nodes that are expanded and since the elimination of nodes is dependent on the graph, we are likely to see much different performance for different graph distributions. We would expect that more nodes are eliminated in cases where there is a large variation in the distances between nodes. However, Branch and Bound must still hit a leaf node before ever eliminating a node which causes the performance to be exponential. In fact, in our testing we found that Branch and Bound failed to produce an optimal solution for cases where $n > 20$.

Table 1: Average Runtime in Seconds Varying k

n (cities)	Number of Distinct Distance Values k	
	k=4	k=8
4	10s	x
5	31s	x
8	1452s	3047s

Table 1 above shows the results of running the branch and bound algorithm on $n = 4, 5$, and 8 cities. We can see an exponential increase in the running time as the number of cities increased. However, as k increased, surprisingly, the running time also increased. We expected the opposite to be true. The reason for the increase in running time when k increases is likely due to the decrease in the amount of nodes that are eliminated when we have an increase in k (we assumed the opposite to be true).

Table 2: Average Runtime in Seconds Varying v

n (cities)	Variance of Distances (S.D.) v			
	v=10	v=50	v=100	v=150
4	11s	21s	36s	47s
5	73s	95s	119s	149s
6	241s	316s	376s	456s
8	3507s	5267s	6928s	8563s

Table 2 above shows the results of running branch and bound on a varying number of cities and a varying standard deviation of the distribution of distances. Surprisingly, as v increases, so does the running time of the algorithm. This is likely again the result of the greater distribution of values leading to less branches being eliminated. The increase in v seems to be linear with the increase in the running time.

The results above suggest that branch and bound works well in cases where there is minimal distribution in the distances. This results to the most amount of branches being eliminated. However, it is likely that in practical scenarios the distribution will be vast and branch and bound will not perform well.

2.7 Future Improvements

The branch and bound algorithm described is unable to scale since it uses recursion and is exponential. We often found that the recursive stack overflows in practice when n is larger than 20. Instead, we could have used an iterative approach to improve the running time of the algorithm in practice. We also could have avoided using custom classes for some calculations and opted to a more efficient simple approach. The extra data we use likely causes the recursive stack to grow faster and causes the runtime to be drastically reduced.

3 Nearest Neighbor Local Search

The second solution to the TSP utilizes the Nearest Neighbor Heuristic to compute an initial solution and a local search method to improve on that solution. The final solution is not guaranteed to be optimal, but in practice it is close to optimal.

3.1 Nearest Neighbor Heuristic

To obtain an initial solution (not optimal), we can use the nearest neighbor heuristic. This heuristic works by choosing the nearest city in the current path. The psuedocode for the nearest neighbor heuristic is shown below:

```
NearestNeighbor(adjMatrix, start, size) {
    availableNodes = new int[size](1);

    current = Node(start, 0, null, start);
    availableNodes[start] = 0;

    for (i = 0; i < size; ++i) {
        if (i != start) {
            min = inf;
            next = i;
        }
    }
}
```

```

    for (j = 0; j < size; ++j) {
        if (j != current && availableNodes[j]
            && adjMatrix[current][j] < min) {
            min = adjMatrix[current][j];
            next = j;
        }
    }

    current = Node(next, 0, current, start);
    availableNodes[next] = 0;
}

return current;
}

```

We start by initializing the available cities as every city in the graph. We then iterate over the graph and find the lowest cost path we can take from the node we are currently at. We then return the path.

3.2 Two Opt Local Search

Given that we have obtained an initial solution using the nearest neighbor heuristic, we can improve our solution by making improvements to the path. One way we can do this is discussed in [2]. Lin and Kernighan discuss a "k opt" solution in which k swaps are made to improve the solution. In particular, we will focus on the two opt algorithm described. The algorithm takes two pairs of vertices and swaps the edges between them to construct a new valid TSP solution. We can calculate the path cost of this new solution and compare it to our best path cost. We update the best path if we find a cheaper solution. This process describes our objective function, the cost of the TSP path. We want to minimize this path by searching local solutions that can improve the cost of the solution. Our state space is a full tour that visits every city once and returns to the origin city. We continue the two opt swapping until we have no longer made an improvement to our best cost. This is why the search is local as it may stop at a local minimum that is not the global optimal path. The pseudocode for the Two Opt swap is shown below:

```

swapTwoOpt(path, i, j) {
    //reverse the path from i to j (Two Opt Swap)
    //Doing a sweep swap from i to j emulates swapping the edges
    //between the pair of vertices
    while (i < j) {
        std::swap(path[i], path[j]);
        i++;
        j--;
    }
}

```

The swapTwoOpt function takes a valid path and the two vertices we would like to swap. We then loop incrementing i and decrementing j until they meet. We swap i and j in each iteration which simulates swapping two edges that construct a valid TSP solution.

3.3 The Complete Algorithm

The Nearest Neighbor Heuristic function requires a starting city and if the TSP problem does not define a starting city (which is what is assumed), we need to run this function for every starting city. Thus, we

need to create a parent function that iterates through all the cities and runs the nearest neighbor heuristic function. Once we find the best nearest neighbor approximation, we will loop through every pair of vertices in the solution and perform a 2 opt swap using the swapTwoOpt function previously defined. If we find a new solution from swapping two nodes that improves the current path, we will perform a 2 opt swap on the new best current path. The psuedocode of our parent function is shown below:

```
std::vector<int> Graph::NearestNeighborTwoOpt() {
    //Find best Nearest Neighbor solution
    bestNN = null;

    for (int i = 0; i < numVertices; i++) {
        NNSolution = NearestNeighbor(M, i, numVertices);
        if (!bestNN || pathCost(NNSolution) + M[originIndex][NNSolution] <
            pathCost(bestNN) + M[originIndex][bestNN]) {
            bestNN = NNSolution;
        }
    }

    bool madeImprovement = true;
    int numImprovements = 0;
    while (madeImprovement && numImprovements < 10000/numVertices) {
        madeImprovement = false;

        for (i = 0; i < numVertices; i++) {
            for (j = 0; j < numVertices; j++) {
                prevCost = calculatePathCost(bestPathNN);
                swapTwoOpt(bestPathNN, i, j);
                if (pathCost(bestPathNN) < prevCost) {
                    ++numImprovements
                    madeImprovement = true;
                    break;
                } else {
                    swapTwoOpt(bestPathNN, i, j);
                }
            }
            if (madeImprovement) {
                break;
            }
        }

    }

    return bestPathNN;
}
```

The algorithm starts by finding the best Nearest Neighbor TSP solution obtained from choosing a new starting city. We then create a bool variable that describes if an improvement has been made on the best path. We iterate through the possible Two Opt Swaps we can make from the current best solution. We compute the path cost of a new path by summing the costs of edges in the path using the adjacency list. If a cheaper cost is found, we set the new best path to that path. This algorithm will find the best solution in its local search area (Starting with the Nearest Neighbor solution).

3.4 Space and Time Complexity

To analyze the time complexity of this algorithm, we will need to analyze every method defined above.

3.4.1 Nearest Neighbor Time Complexity

The Nearest Neighbor Heuristic iterates through every edge in the graph. Therefore, the runtime of it is $O(E) = O(N \cdot N - 1) = O(N^2)$. However, since we do not have a starting city, we iterate through every starting city, causing the runtime to become $O(N^3)$.

3.4.2 Two Opt Time Complexity

The time complexity of the Two Opt part of the algorithm is hard to represent since we do not know when an improvement will fail to be made. The algorithm stops when no improvement is made. This means in the worst case we could iterate through every possible path permutation, resulting in 2^n different iterations. However, in each iteration we need to find the best improvement among all the pair of edges. This takes $O(N^2)$ time since there are $N \cdot (N - 1)$ edges in the graph. Therefore, the worst case time complexity of the Two Opt portion of this algorithm is $O(N^2 \cdot 2^n)$. However, we will find that the Two Opt algorithm converges much quicker than this in most cases.

3.4.3 Total Time Complexity

If we combine the time complexity of both the Nearest Neighbor Heuristic and the Two Opt Algorithm, we will obtain a total time complexity of $O(N^3 + N^2 \cdot 2^n) = O(N^2 \cdot 2^n)$. Although this time complexity is not much better than the previously discussed optimal algorithm, in most practical scenarios, this is drastically reduced. We can also inject an upper bound in the algorithm by limiting how many times we search for a better solution. This will reduce the optimality of the algorithm but will drastically improve the runtime.

3.4.4 Space Complexity

The Nearest Neighbor Heuristic does not introduce new space. It utilizes existing space of the adjacency matrix (which is a space of N^2) and returns a path that is of size $N + 1$. The Two Opt Algorithm also does not introduce new space. It rather uses the existing space of the adjacency matrix. Therefore, this algorithm has a relatively low space complexity of $O(N^2)$ that only consists of the graph.

3.5 Experimental Results

In practice Nearest Neighbor Two Opt performs exceptionally well. That is, it is able to come up with a solution in a small amount of time. However, the optimality of the solution is to be questioned. Since we implemented an upper bound on the amount of improvements are made by the algorithm, the algorithm terminates before it can become exponential. The upper bound we made is $10,000/n$, where n is the number of cities in the TSP problem. This ensures that in cases where we have many cities (for example 1000), we do not attempt to make many improvements (maximum of 10). The reason for this is each improvement takes n^2 time to find, which means in the case of 1000 cities we need to make 1,000,000 iterations before terminating an improvement.

It is difficult to determine if a tour is optimal. It is also difficult to deduce the optimality of a given tour. Since the TSP is in NP-Hard, we do not know if there exists a polynomial time algorithm that determines if a given tour is optimal. However, in situations where $n < 20$, we see that two opt returns the optimal solution in most cases.

Table 3: Average Number of Iterations of SLS on TSP

n (cities)	Number of Distinct Distance Values k		
	k=5	k=10	k=100
5	25	x	x
25	872.8	1,257.6	x
100	14,166.6	16,016.2	25,293.8
1000	1,537,539	1,161,132	247,580.5

Table 3 above shows the number of iterations on average were needed for two opt to terminate in various scenarios. We can see that generally an increase in the number of distinct distance values, we see an increase in number of iterations for two opt. However, once we reach $n = 1000$ we notice the opposite pattern. This is likely due to the upper bound imposed by our algorithm to the number of improvements that can be made. For $n = 1000$, only 10 improvements can be made before the algorithm terminates. As k increases, improvements will likely be made earlier (since distinct values causes large differences in a given tour cost).

Although it is difficult to determine the optimality of each tour solution produced by SLS, we can compare the tours produced by the complete Heuristic Search to the tours produced by SLS. This is only possible when n is small because of the exponential time complexity of the complete algorithm.

Table 4: Percent Error Between SLS and HS

n (cities)	Case		
	Greatest Error	Least Error	Average Error
4	20.71%	0%	3.38%
8	58.01%	0%	15.18%

Table 4 above shows the percent error seen between the tours produced by SLS and HS (That is, we compute the optimality of the tour produced by SLS). We found that in many cases we see a low error (below 5%) and in many cases we saw that the optimal tour length was produced. However, in some cases we found that the error can be drastic. For example, with 8 cities, we found a drastic error of 58.01%. This is due to the SLS starting from a tour that has a local minimum that is 58.01% worse than the optimal tour. The overall average error we found was relatively small. We found that as the number of cities grew, the average error drastically grew. This could be because of the upper bound on the number of improvements that can be made or it could also be because as we have more cities there are more tours that can be produced (a bad local minimum is much more possible).

In testing we found no significant impact on the error from the variables k , u , or v . It is possible that these variables have some impact on the tour error but because of HS's exponential performance is difficult to test.

3.6 Improvements made and Future Improvements

One improvement we made to the SLS algorithm was to add an upper bound on the number of two opt improvements that can be made. This bound is reduced as the number of cities n increases. The choice of using $10,000/n$ for the upper bound was largely due to the fact that once n reaches 10,000 it is too expensive to compute a valid TSP tour. This allows TSP to find a reasonably good solution without timing out.

Another angle we could have gone using SLS could be using probability theory to only accept swaps in certain scenarios (such as the Simulated Annealing Algorithm discussed in [4]). This would have likely improved the optimality of a tour we find using SLS by avoiding staying in a local search space (We avoid bad

local minimums).

4 Conclusion

This paper discussed two different solution to the Travelling Salesman Problem. The first solution makes use of the branch and bound method and uses a lower bound heuristic to prune the search space. This solution is complete and optimal but has an exponential runtime and an exponential space complexity. The second solution uses the Nearest Neighbor Hueristic to obtain an initial solution to the TSP and improves the solution using a local search algorithm, Two Opt. The algorithm swaps pairs of edges to see if a better solution is possible. The second solution also has exponential worst case time complexity but it is much less in practice. It also has a low space complexity that is limited to the graph representation. However, the second solution returns a solution that is close to optimal but not optimal. Both these solutions encompass the trade offs of solutions to the TSP. One could focus on optimality, but sacrifice runtime or one can accept a near optimal solution with much better runtime in practice.

We ran multiple experiments and tests on the two algorithms described. We found that BnB struggles to produce an optimal tour when $n > 20$. We also found that the both variables k (distinct number of values for distances) and v (standard deviation of the distribution of distances) have a significant effect on the running time of the algorithm. As k and v increase, so does the running time of the branch and bound algorithm. On the other hand, the two opt algorithm was not affected by the variables k or v . Additionally, two opt was able to easily solve all the problems presented in the respectable number of iterations. However, two opt struggled with its solution quality, where as the number of nodes increased, the overall quality of the solution decreased.

5 References

- [1] Balas Egon, and Toth Paolo. "Branch and Bound Methods for the Traveling Salesman Problem." Carnegie-Mellon University, MSRR 488. <https://apps.dtic.mil/sti/tr/pdf/ADA126957.pdf>.
- [2] Lin, S., and B. W. Kernighan. "An Effective Heuristic Algorithm for the Traveling-Salesman Problem." *Operations Research*, vol. 21, no. 2, 1973, pp. 498–516. JSTOR, <http://www.jstor.org/stable/169020>. Accessed 18 Feb. 2024.
- [3] Flood, Merrill M. "The Traveling-Salesman Problem." *Operations Research*, vol. 4, no. 1, 1956, pp. 61–75. JSTOR, <http://www.jstor.org/stable/167517>. Accessed 18 Feb. 2024.
- [4] Zhan SH, Lin J, Zhang ZJ, Zhong YW. List-Based Simulated Annealing Algorithm for Traveling Salesman Problem. *Comput Intell Neurosci*. 2016;2016:1712630. doi: 10.1155/2016/1712630. Epub 2016 Mar 13. PMID: 27034650; PMCID: PMC4808530.

Appendix A

Competition Results

Table 5: Some Results from Competition

File	Tour Cost	Runtime	Iterations
tsp-problem-25-125-100-25-1.txt	1558.5	0	625
tsp-problem-25-125-100-5-1.txt	2291.29	1	1594
tsp-problem-25-250-100-25-1.txt	1430.99	0	1273
tsp-problem-25-250-100-5-1.txt	2303.37	2	2919
tsp-problem-25-31-100-25-1.txt	1393.48	0	917
tsp-problem-25-31-100-5-1.txt	2299.69	1	2072
tsp-problem-25-6-100-25-1.txt	1424.87	1	1344
tsp-problem-25-6-100-5-1.txt	2462.85	0	625
tsp-problem-25-62-100-25-1.txt	1568.54	1	1293
tsp-problem-100-100-100-25-1.txt	4306.74	46	25656
tsp-problem-100-100-100-5-1.txt	8964.53	65	36053
tsp-problem-100-1000-100-25-1.txt	4424.11	23	12199
tsp-problem-100-1000-100-5-1.txt	8873.14	55	29774
tsp-problem-100-2000-100-25-1.txt	4691.35	96	53724
tsp-problem-100-2000-100-5-1.txt	8887.97	114	64075
tsp-problem-100-4000-100-25-1.txt	4516.97	34	19228
tsp-problem-100-4000-100-5-1.txt	8900.41	90	52005
tsp-problem-100-500-100-25-1.txt	4571.01	90	49772
tsp-problem-100-500-100-5-1.txt	8975.45	74	40986
tsp-problem-200-16000-100-25-1.txt	7796.42	388	123740
tsp-problem-200-16000-100-5-1.txt	17612.6	248	81503
tsp-problem-200-2000-100-25-1.txt	8721.51	568	184234
tsp-problem-200-2000-100-5-1.txt	17596.7	510	171841
tsp-problem-200-400-100-25-1.txt	7122.27	883	282806
tsp-problem-200-400-100-5-1.txt	17728.8	451	148721
tsp-problem-200-4000-100-25-1.txt	7875.01	508	162318
tsp-problem-200-4000-100-5-1.txt	17613	237	69632
tsp-problem-200-8000-100-25-1.txt	7873.85	391	121156
tsp-problem-200-8000-100-5-1.txt	17577.6	365	122204
tsp-problem-1000-10000-100-25-1.txt	27073	23353	782556
tsp-problem-1000-10000-100-5-1.txt	85297	10729	316946
tsp-problem-1000-100000-100-25-1.txt	26132.6	22462	746233
tsp-problem-1000-100000-100-5-1.txt	85246.6	23363	780648
tsp-problem-1000-200000-100-25-1.txt	26312.8	12838	383038
tsp-problem-1000-200000-100-5-1.txt	85337.1	10522	316033
tsp-problem-1000-400000-100-25-1.txt	26817.8	19647	693504
tsp-problem-1000-400000-100-5-1.txt	85176.8	13957	435340

Table 5 above shows some results from running the competition benchmark. These results show the tour cost, the runtime, and the number of iterations. Note that these results are from running the SLS algorithm. The BnB algorithm failed to perform on these problems.